NHIỀU CỬA SỔ & FRAME LỒNG NHAU

Selenium + pytest

Nhóm W4 - N07

Thành viên

Nguyễn Tiến Toàn - A51878
Nguyễn Trung Đức - A52093
Đào Quang Khải - A51922
Nguyễn Anh Tuấn - A51709
Phạm Bảo Phúc - A52197

MỤC TIÊU & TRANG THỰC HÀNH

MỤC TIÊU

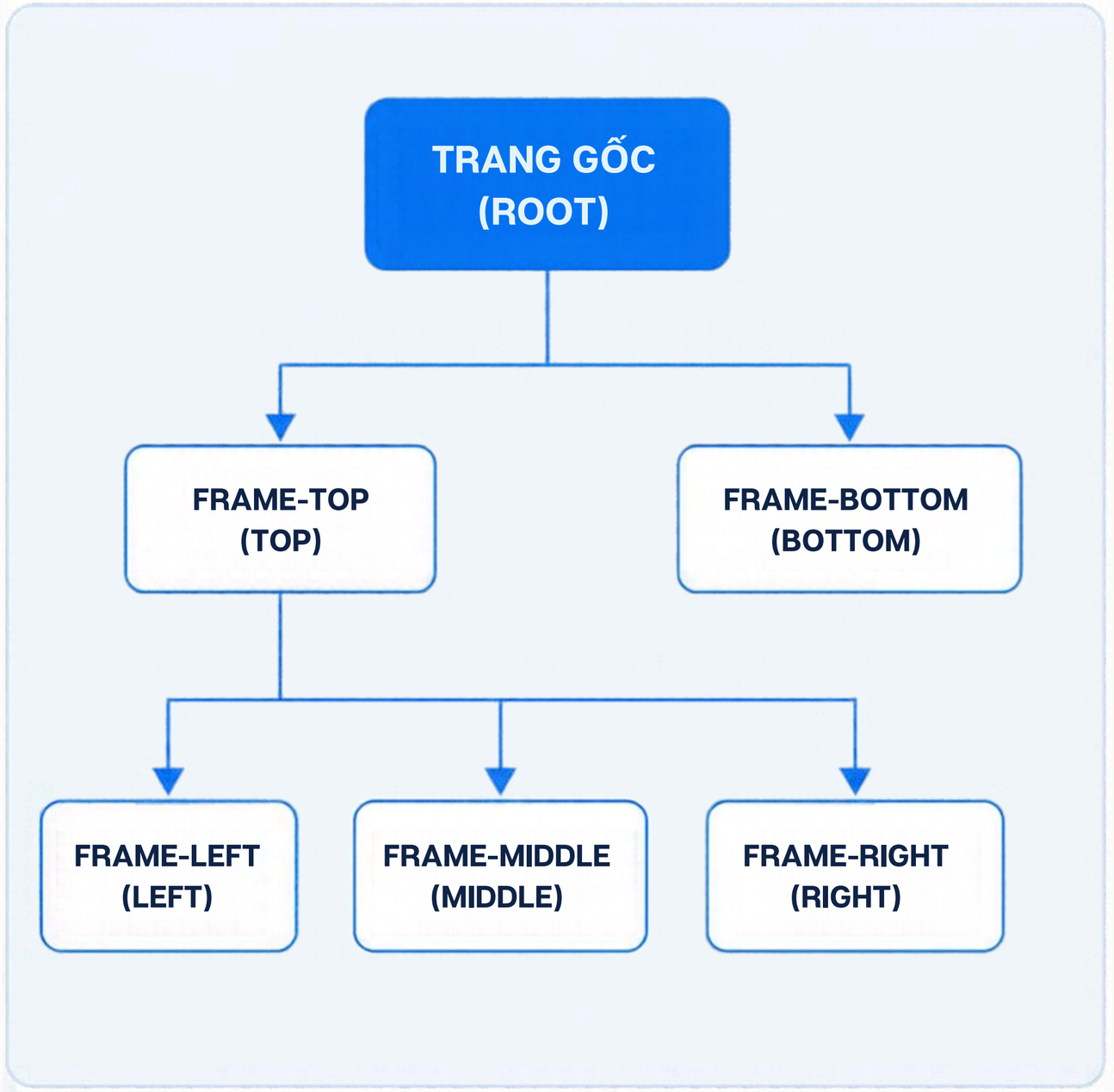
Điều khiển nhiều cửa sổ/tab và frame lồng nhau băng Selenium

TRANG THỰC HÀNH

the-internet.herokuapp.com/windows
the-internet.herokuapp.com/nested_frame

TÀI LIỆU THAM KHẢO

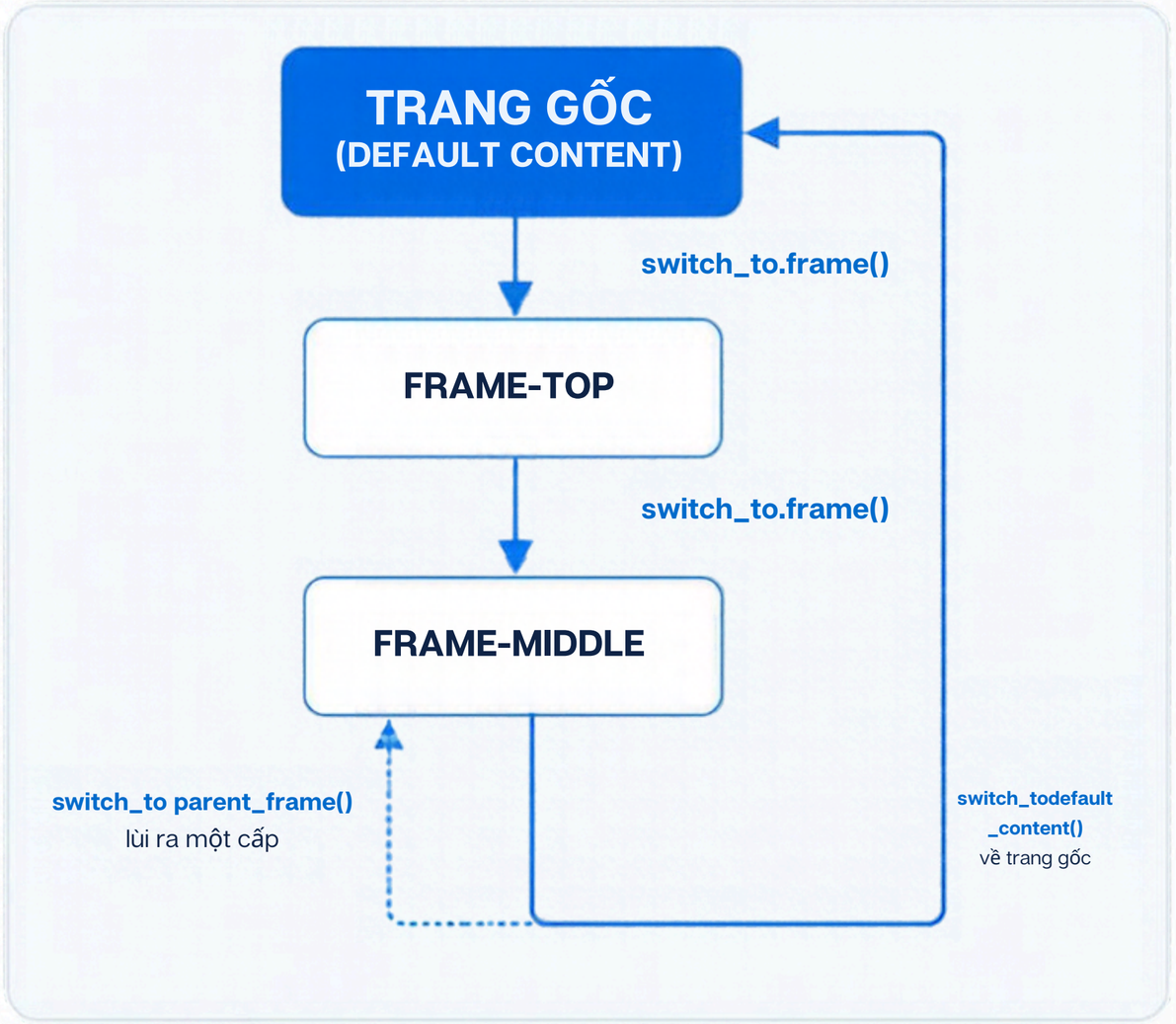
3.4, 3.6, 3.7, 7.19, 5.1, 7.1 (selenium-python)



NHIỀU CỬA SỔ

HÀM	Ý NGHĨA
WINDOW_HANDLES	LẤY DANH SÁCH ID CỦA TẤT CẢ CỬA SỔ/TAB ĐANG ĐƯỢC WEBDRIVER QUẢN LÝ
CURRENT_WINDOW_HANDLE	LẤY ID CỦA CỬA SỔ/TAB HIỆN TẠI
SWITCH_TO.WINDOW(HANDLE)	CHUYỂN SANG CỬA SỔ/TAB CÓ ID TƯƠNG ỨNG
EC.NUMBER_OF_WINDOWS_TO_BE()	CHỜ SỐ LƯỢNG CỬA SỔ/TAB BẰNG N

FRAME LỒNG NHAU – MỤC 3.4



HÀM	MÔ TẢ
switch_to.frame(x)	Chuyển vào frame x trước khi tìm phần tử bên trong
switch_to parent_frame()	Lùi ra một cấp về frame cha (nếu có)
switch_todefault_content()	Về thẳng trang gốc (default content), thoát khỏi mọi frame

THÔNG TIN CƠ BẢN VỀ WAITS

1. Tổng quan về Waits

Mục đích: Đồng bộ tốc độ xử lý của kịch bản kiểm thử (test script) với tốc độ tải/phản hồi của trang web.

Tác dụng: Tránh lỗi không tìm thấy phần tử (NoSuchElementException) khi giao diện chưa kịp tải xong.

3 loại Waits chính: Implicit Wait, Explicit Wait và Fluent Wait.

BẢNG SO SÁNH 3 LOẠI WAIT TRONG SELENIUM	
Implicit Wait (Chờ ngầm định)	Chờ phần tử xuất hiện trong DOM (Presence)
Explicit Wait (Chờ tường minh)	Chờ theo trạng thái cụ thể (Visible, Clickable, Frame available,...)
Fluent Wait (Chờ linh hoạt nâng cao)	Chờ theo điều kiện linh hoạt + tùy chỉnh tần suất lặp/bỏ qua lỗi.

EXPLICIT WAITS & EXCEPTIONS – MỤC 5.1, 7.1

EXPLICIT WAITS

Là cơ chế chờ trong Selenium, cho phép chương trình đợi đến khi một điều kiện cụ thể được thỏa mãn hoặc hết thời gian chờ mới tiếp tục thực hiện.

EXCEPTIONS THƯỜNG GẶP	
NoSuchFrameException	sai tên frame hoặc quên default_content()
NoSuchElementException	tìm phần tử khi chưa vào frame
NoSuchWindowException	dùng driver sau close()
TimeoutException	hết giờ chờ

HISTORY, COOKIES, REMOTE WEBDRIVER - MỤC 3.6, 3.7, 7.19

HISTORY

> back() / forward(): Điều hướng trang

> current_url: Lấy URL đang mở

COOKIES

> add_cookie / get_cookie

> cookie dùng chung giữa các tab cùng domain

REMOTE WEBDRIVER

```
webdriver.Remote(
  command_executor='http://host:4444'
)
```

> chạy trên Selenium Grid

> Code test giữ nguyên

> hỗ trợ đầy đủ API windows, frames, cookies giống local driver.

CODE TEST CỦA NHÓM

```python
# Xử lý Windows & Explicit waits (3.4. Windows and frames & 5.1. Explicit Waits)
def test_windows_handling():
    driver = webdriver.Chrome()
    driver.get("https://the-internet.herokuapp.com/windows")

    # Khởi tạo bộ đếm thời gian chờ tối đa 10 giây
    wait = WebDriverWait(driver, 10)

    # Lưu lại ID của cửa sổ gốc
    original_window = driver.current_window_handle

    # Click mở tab mới
    driver.find_element(By.LINK_TEXT, "Click Here").click()

    # Chờ đến khi tổng số cửa sổ mở ra là 2
    wait.until(EC.number_of_windows_to_be(2))

    # Lấy danh sách toàn bộ ID của các tab đang mở
    for window in driver.window_handles:
        if window != original_window:
            # Chuyển con trỏ sang tab mới
            driver.switch_to.window(window)
            break

    element = driver.find_element(By.TAG_NAME, "h3")
    assert element.text == "New Window"

    driver.quit()
```

Xử lý Windows & Explicit waits
(Windows and frames & Explicit Waits)

```python
# Xử lý Frames (3.4. Windows and frames)
def test_frames_handling():
    driver = webdriver.Chrome()
    driver.get("https://the-internet.herokuapp.com/nested_frames")

    # Chuyển vào frame frame-top
    driver.switch_to.frame("frame-top")

    # Từ frame-top, đi tiếp vào frame con của nó (frame-middle)
    driver.switch_to.frame("frame-middle")
    assert driver.find_element(By.ID, "content").text == "MIDDLE"

    # Lùi lại 1 cấp (Từ middle lùi về top)
    driver.switch_to.parent_frame()

    # Thoát ra khỏi mọi frame, về thẳng HTML gốc
    driver.switch_to.default_content()

    driver.quit()
```

Xử lý Frames
(Windows and frames)

CODE TEST CỦA NHÓM

```python
# Kết hợp Wait và Frame (3.4. Windows and frames & 5.1. Explicit Waits)
def test_frame_wait_and_switch():
    driver = webdriver.Chrome()
    driver.get("https://the-internet.herokuapp.com/nested_frames")

    # Cài đặt thời gian chờ
    wait = WebDriverWait(driver, 10)

    # Đợi frame-bottom xuất hiện, có mặt là tự động switch_to.frame
    wait.until(EC.frame_to_be_available_and_switch_to_it("frame-bottom"))

    assert "BOTTOM" in driver.page_source
    driver.quit()

# Lịch sử và điều hướng (3.6. History and location)
def test_history_and_location():
    driver = webdriver.Chrome()

    # Truy cập trang này, sau đó truy cập trang sau
    driver.get("https://the-internet.herokuapp.com/windows")
    driver.get("https://the-internet.herokuapp.com/")

    # Nhấn nút Back trên trình duyệt để về trang trước đó
    driver.back()
    assert "windows" in driver.current_url

    # Nhấn nút Forward để tiến tới trang sau
    driver.forward()
    assert driver.current_url == "https://the-internet.herokuapp.com/"

    driver.quit()
```

Kết hợp Wait và Frame
(Windows and frames & Explicit Waits)
Lịch sử và điều hướng
(History and location)

```python
# Quản lý Cookies (3.7. Cookies)
def test_cookies():
    driver = webdriver.Chrome()
    driver.get("https://the-internet.herokuapp.com/windows")

    # Thêm một cookie mới vào trình duyệt
    driver.add_cookie({'name': 'my_test_cookie', 'value': '12345'})

    # Lấy toàn bộ cookies hiện có
    cookies = driver.get_cookies()

    # Kiểm tra xem cookie vừa thêm có tồn tại không
    assert any(c['name'] == 'my_test_cookie' and c['value'] == '12345' for c in cookies) # vừa

    driver.quit()

# Bắt lỗi (7.1. Exceptions)
def test_exceptions():
    driver = webdriver.Chrome()
    driver.get("https://the-internet.herokuapp.com/windows")

    wait = WebDriverWait(driver, 2)

    # Dùng try-except để chương trình không bị sập khi gặp lỗi
    try:
        # Cố tình chờ 1 element không tồn tại để ép văng lỗi
        wait.until(EC.presence_of_element_located((By.ID, "id_khong_ton_tai")))
    except TimeoutException:
        # Bắt đúng loại lỗi TimeoutException
        print("Đã bắt được lỗi do hết thời gian chờ.")
        pass

    driver.quit()
```

Quản lý Cookies
Bắt lỗi (Exceptions)

CODE TEST CỦA NHÓM

```python
# Remote WebDriver (7.19. Remote WebDriver)
def test_remote_webdriver():
    # Phải có Selenium Server đang chạy ở port 4444 thì đoạn này mới hoạt động
    try:
        options = webdriver.ChromeOptions()
        # Thay vì driver = webdriver.Chrome(), ta kết nối tới máy chủ từ xa
        driver = webdriver.Remote(
            command_executor='http://127.0.0.1:4444/wd/hub',
            options=options
        )
        driver.get("https://the-internet.herokuapp.com/windows")
        assert "The Internet" in driver.title
        driver.quit()
    except Exception:
        print("Bỏ qua bài test này vì không có Selenium Server port 4444")
        pass
```

Remote WebDriver

FRAME LỒNG NHAU – MỤC 3.4

Kết quả chạy thử

```
PS C:\Users\Admin\Documents\A51878\Kiểm_thử\my_selenium_project> pytest test_windows_nestedframes.py
============================= test session starts ==============================
platform win32 -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Admin\Documents\A51878\Kiểm_thử\my_selenium_project
collected 7 items

test_windows_nestedframes.py .......                                    [100%]

============================== 7 passed in 78.84s (0:01:18) ==============================
PS C:\Users\Admin\Documents\A51878\Kiểm_thử\my_selenium_project>
```

Cảm ơn cô và các bạn đã lắng nghe

THANK YOU FOR LISTENING